

Meta Model

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

Who this book is for . . . 2

A book that stands on its own . . . 3

What you need . . . 3

Conventions . . . 3

I: The Why . . . 5

1. One definition, many spellings . . . 6

1.1. A published language . . . 6

1.2. Publishing is the deliverable . . . 6

1.3. Agents need structure . . . 6

1.4. The incumbent is a spreadsheet . . . 7

2. Documentation as structure . . . 8

2.1. Where each element came from . . . 9

3. What Meta leaves out . . . 10

3.1. Annotations, the bargain Ecore makes . . . 10

3.2. Specializations per target . . . 10

3.3. Why the JVM is not built on Meta . . . 11

4. Federated by design . . . 12

4.1. Directly on the core model . . . 12

4.2. The landscape . . . 12

II: The How . . . 14

5. Getting started . . . 15

5.1. Adding the model . . . 15

5.2. Names that clash . . . 16

5.3. What the build does . . . 16

6. The model . . . 17

6.1. Base . . . 17

6.2. Structure . . . 17

6.3. Members . . . 18

6.4. Types . . . 18

6.5. Instances . . . 19

7. A vocabulary . . . 20

7.1. Builders . . . 20

7.2. The vocabulary . . . 21
7.3. Saving and loading . . . 22
8. The text syntax . . . 23
8.1. A package and its imports . . . 23
8.2. Data types and enumerations . . . 24
8.3. Classes . . . 24
8.4. Loading . . . 27
8.5. Federation . . . 28
8.6. Facets . . . 29
8.7. Problems and provenance . . . 29
8.8. Validating a directory . . . 30
8.9. Saving . . . 31
8.10. The bank's glossary . . . 33
9. Projections . . . 34
9.1. A glossary, for people . . . 34
9.2. A dynamic Ecore package, for tools . . . 36
9.3. What a projection decides . . . 37
10. Instances whose type is a reference . . . 39
10.1. The rule that keeps it small . . . 39
10.2. Classified later . . . 40
10.3. What it makes possible . . . 40
10.4. Status . . . 40
11. AI-assisted domain design . . . 42
11.1. Who checks what . . . 42
11.2. How it goes . . . 43
11.3. Reading a vocabulary without learning the notation . . . 43
11.4. What to watch for . . . 45
11.5. The agent skill . . . 45
11.6. A vocabulary project . . . 46
III: Looking Ahead . . . 49
12. Formats . . . 50
12.1. Authoring formats . . . 50
12.2. Hybrid authoring . . . 51
12.3. Output formats . . . 52
12.4. Who uses which . . . 53
13. Looking ahead . . . 54

13.1. Flattening . . . 54
13.2. Variability . . . 55
13.3. Occupancy . . . 55
13.4. Kinds and types . . . 56
Appendix A: Reference . . . 57
A.1. Classifiers . . . 57
A.2. Text syntax . . . 57
A.3. Primitive kinds . . . 58
A.4. Multiplicity . . . 59
Appendix B: Building this book . . . 60
Appendix C: Glossary . . . 61

Colophon

Meta Model: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the Meta model of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime.models.meta:model:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/meta`. Every listing in it is included from the module's tests, which are compiled and run when the book is built.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Meta Model: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Eclipse and EMF are trademarks of the Eclipse Foundation. Java is a trademark of Oracle and its affiliates. TypeScript is a trademark of Microsoft Corporation. Excel is a trademark of Microsoft Corporation. draw.io is a trademark of JGraph Ltd. Other names may be trademarks of their owners. Their use here identifies the technologies the model relates to and implies no endorsement.

The source of this book, its tests and the model it describes are at github.com/Nasdanika-SDK/runtime, in `models/meta/model`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about Meta, the language-neutral type system of the Nasdanika SDK Runtime: a micro-model of typed domain models, with packages, classes, features, operations and the type references between them, in about twenty classifiers. A vocabulary is written down once in Meta, with its documentation as structure, and projected to whatever its consumers need: a glossary or a PDF for people, a dynamic Ecore package for tools, TypeScript for browsers, JSON Schema for the tools of agents. Developers write it in Meta's text syntax, `.nmeta` files. The people who own the words can have an AI assistant write it for them today, with the build checking the structure and themselves the meaning; they will also write it in the spreadsheets, diagrams and Markdown they already use.

The first part of the book is the why: why one definition should not live in many formats, why documentation should be structure, what Meta leaves out on purpose, and why there is no server. The second part is the how, for Java developers: a vocabulary of a library, built in Java; the vocabulary of a bank, with its customers, products, accounts, transactions and statements, written in the text syntax and loaded through a resource set; and projections of both to a glossary and to Ecore. It ends with domain design with an AI assistant: who checks what, how a subject-matter expert works with one, the validator and the agent skill. The third says what comes next: the formats vocabularies will be authored and published in, with who each is for, flattening, and vocabularies that vary by where and when they apply.

Who this book is for

Architects and modelers who keep a domain's vocabulary, today in spreadsheets, wiki tables and schemas that disagree, and want it in one place with the types and the documentation generated from it.

Developers who write vocabularies in the text syntax, and projections from a vocabulary to code, schemas, documentation, or the context of an agent.

Business owners of a vocabulary and other subject-matter experts, who will read Part I, the first half of Chapter 11, which is how they can take part today with no code, and Chapter 12: what they will author in, and what they will get back.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. You do not need any other book of the series, or the Nasdanika SDK Runtime book, to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The model's Java module is `org.nasdanika.sdk.runtime.models.meta`, its Maven coordinates are `org.nasdanika.sdk.runtime.models.meta:model:2026.11.0`, and its namespace URI is `runtime.sdk.nasdanika.org/models/meta`.
- Some familiarity with EMF helps in part two: an EMF model is a set of Java interfaces generated from a metamodel, with containment, references, reflection and persistence for free. Meta is itself an EMF model, and one of its projections is to Ecore, EMF's metamodel.

Conventions

- **Every listing is included from a test** of the module and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Status labels** mark what this edition does not have yet: **PLANNED** for designed but not built, **SKETCH** for partly built.
- **Voice.** I write in the first person.



> Editor's notes record inconsistencies between the model and its documentation found while writing. They are shown in builds for the author and left out of published editions.
>
> This book replaces the model's readme, `models/meta/model/doc/readme.md`, which is deleted. The readme placed Meta in a model tower with floors and an occupancy model; the runtime has neither yet, and this book describes what it has, with the

> rest as planned. The Xcore's comments still say "nxcore" for the core model.
>
> The example in the header of `Meta.g4` throws `Error`, which it does not declare, so the reader reports an unknown type; the tests declare it. The grammar's header says a facet is attached to its element as a contained object; the core model's `facets` feature is commented out, so the reader records facets as annotations until it is not (Section 8.6).

Part I: The Why

Why a vocabulary should be written down once, why its documentation should be structure, what Meta leaves out, and why there is no server. No code is needed to read this part.

Chapter 1. One definition, many spellings

A class with named, typed, possibly multi-valued features is one of the most stable ideas in computing, and every ecosystem spells it its own way: Ecore, Java, XML Schema, JSON Schema, a database schema, TypeScript, a table on a wiki page, a spreadsheet. The differences are real and small, and most of them are about the host language, not about the domain.

The cost is not the spelling. It is that one definition, of a customer, an order or a service, lives in many formats at once and nothing relates them: a DDL script, a JSON Schema, a partner's XSD, a Java class, a wiki table and a few spreadsheets. Each is a projection of a domain model that was never written down. Meta is the place to write it down once, and project.

1.1. A published language

In the terms of Domain-Driven Design, Meta is a Published Language: a documented, shared vocabulary that bounded contexts translate into and out of. It adds the part that was out of reach when the term was coined: the vocabulary is machine-checkable, because the types and the documentation are generated from it.

It is deliberately not a superset of any of its sources. Whatever does not fit is dropped by design or carried in an annotation, and the loaders decide which (Chapter 3).

1.2. Publishing is the deliverable

A model that cannot be read by the people who own the vocabulary is a private notation. A vocabulary published as a site, with a page per element, its documentation, its neighbors and a diagram of its neighborhood, is a glossary that cannot drift from the types, because both come from one source. That is what a business stakeholder opens, and nobody has to hear the word "metamodel".

1.3. Agents need structure

An assistant reasoning over an organization's systems has to know what a customer is, what an account is, and how they relate. Today that context is assembled by hand, per project, from whatever prose can be found. An agent given a declared, versioned, owned vocabulary answers

from it and cites it; an agent given a DDL script and a wiki export infers, and inference is not a chain of custody.

1.4. The incumbent is a spreadsheet

The competitor is not another modeling tool. It is a spreadsheet and a wiki table, which win because they are free, everyone can use them, and they never ask anyone to adopt anything. Meta treats them as inputs, not as enemies: the vocabulary is captured where it already lives, and projected from there **PLANNED**. Developers get a text syntax of their own, and everybody gets the outputs they read, down to a PDF for a partner (Chapter 12). Until those inputs are here, the people who own the words can have an assistant write the text syntax for them, and check what it says about their business (Chapter 11).

Chapter 2. Documentation as structure

Most formats carry documentation as one string, and some do better. The claim worth making is narrower than "nobody supports documentation":

Source	Documentation it carries
Ecore	One string, in a GenModel annotation detail
Java	Nothing reflective: Javadoc lives in source comments, not in the type model
JDBC	`REMARKS`, one string per table or column, often empty
JSON Schema	A `title` and a `description`
Protocol Buffers	Source comments, and only when descriptors are built to keep them
XML Schema	`xs:annotation/xs:documentation`: repeatable, tagged with a language, markup permitted
OpenAPI	`description` in CommonMark, and `externalDocs`
OWL and RDFS	`rdfs:comment`, `skos:definition`, extensible and tagged with a language
Meta, through the core model	`documentation`, `docFormat`, `docRef`, `docContents`, `docSections`

XML Schema, OpenAPI and OWL do respectably. None of them treats documentation as structure. A string cannot be divided into sections, cannot point to a document that lives elsewhere, and cannot declare its own format.

Every Meta element is an element of the runtime's core model, so its documentation can do all three:

- `documentation` is text, in the format `docFormat` names, such as Markdown;
- `docRef` points to a document elsewhere;
- `docContents` and `docSections` structure it, so a glossary entry can have a definition, examples and notes for implementers as separate sections.

2.1. Where each element came from

Core model elements also have markers: where an element came from, a file and a position, or a Git repository, branch and commit. For a vocabulary, markers are the provenance story: this class from that sheet of that spreadsheet, at that commit; this feature from that element of that schema. When two sources disagree about what a term means, the model shows both, and where each came from.

Elements also have `uris`, alternative identifiers that let vocabularies maintained apart be assembled into one, and an `icon`.

Chapter 3. What Meta leaves out

The rule is that what a class-based type system cannot express is not a class in Meta. That keeps Meta at about twenty classifiers, against Ecore's fifty-three, of which a loader at run time uses a few:

- **No `volatile`, `transient`, `unsettable` or `resolveProxies`**: implementation concerns of EMF with no meaning in a browser.
- **No feature maps, no factories, no algebra of generic types.** Unions, intersections and wildcards are carried, so that a Java or TypeScript signature can be written down without loss, but nothing computes subtyping or substitution with them.
- **No subpackages**, which neither TypeScript modules nor npm packages need. A loader flattens them and records the original path.
- **No interface flag.** Whether a class becomes an interface, a type alias or a class is the projection's decision, not a property of the model.

3.1. Annotations, the bargain Ecore makes

Everything else arrives through an `Annotation`, an `OpaqueTypeReference` or a model that extends Meta. An annotation is keyed by a source URI that names its owner, the bargain Ecore's `EAnnotation` makes, so two loaders never collide. Loaders record what they cannot map instead of dropping it, so a round trip to the source stays possible even though Meta is a projection of it.

3.2. Specializations per target

A projection sometimes needs to say something the neutral model should not carry. A Java projection adds a package name and a superclass; a persistence projection adds a table name; a TypeScript projection adds nothing at all. Those live in models that extend Meta, such as a `JavaClass` that extends `Class`, when they are structural enough to deserve types, and in annotations when they are not. The neutral model stays neutral either way, which is what lets one vocabulary serve many projections.

3.3. Why the JVM is not built on Meta

The runtime's JVM model is the test of the rule. Bridge methods, synthetic accessors, access flags, and overloads that differ only in erased parameter types are realities of the JVM that Meta will not grow to hold. So the JVM model does not extend Meta; it projects to it **PLANNED**. A published language is a target of projections, not a base class.

Chapter 4. Federated by design

There is no server and no central repository. A vocabulary is files in Git, or an artifact published to Maven Central or npm, and packages reference each other by namespace URI. A team owns its vocabulary and refers to another team's without coordinating a release, and gets what a catalog cannot give:

- **Branching.** A proposed change to a term, reviewed as a pull request, with a diff that shows which features moved.
- **History.** The same history the code has, with the same tools.
- **Ownership at the edges.** Nobody administers the whole graph.
- **Immutable snapshots.** What a term meant last year is an install, not an investigation.

4.1. Directly on the core model

Meta sits directly on the runtime's core model, and on nothing else. What it needs from below is identity, documentation as structure and markers. That is deliberate: Meta is the foundation of every package generated from it, and a browser application should not drag a governance model in to render a list of customers, nor an agent pay for vocabulary it does not use in its context window on every call. Richer variants extend Meta, not the other way around.

4.2. The landscape

Camp	Examples	What it leaves undone
Industry data models	Licensed enterprise data models	Content you license, not a vocabulary you own; thousands of entities
Standards	schema.org, FHIR, FIBO, ISO 20022	Good content, delivered as OWL files, PDFs and spreadsheets with little tooling. Sources for Meta, not rivals
Data catalogs	Collibra, Alation, Atlan	Organized around where data lives; the meaning stays thin, and the graph is central and rented

Camp	Examples	What it leaves undone
Semantic layers	dbt, Cube, LookML	Analytics only: metrics, dimensions and joins, with no ownership or lifecycle
Ontology platforms	Palantir Foundry	Has the whole thing and keeps it: the ontology does not leave the platform
Modeling tools	Sparx EA, erwin, MagicDraw	The desktop era, weak federation, and the output is a document
The actual incumbent	Spreadsheets and wiki tables	Drift within weeks, cannot be queried, and are invisible to an agent

What is different here is not the part about classes and features, which everybody has. It is the combination: composition with the other models of the runtime, federation without a server, documentation as structure, and portability, a vocabulary reaching a browser and an agent's context as a typed package, without a JVM and without a platform.

Part II: The How

Meta in Java: adding it to a project, its classes, a vocabulary built in Java, a bank's vocabulary written in the text syntax and validated, two projections, and instances whose type is a reference. The last chapter is also for subject-matter experts: domain design with an AI assistant, an agent skill and the build.

Chapter 5. Getting started

Meta is an EMF model, defined in Xcore and generated into Java when the runtime is built.

5.1. Adding the model

The runtime's models have a bill of materials, `org.nasdanika.sdk.runtime.models:bom`, which manages their versions. Import it in `dependencyManagement`, and depend on the model without a version:

```
<dependency>
    <groupId>org.nasdanika.sdk.runtime.models.meta</groupId>
    <artifactId>model</artifactId>
</dependency>
```

The model is a named Java module:

Listing 1. The module declaration of the Meta model

```
module org.nasdanika.sdk.runtime.models.meta {

    exports org.nasdanika.sdk.runtime.models.meta;
    exports org.nasdanika.sdk.runtime.models.meta.impl;
    exports org.nasdanika.sdk.runtime.models.meta.resource;
    exports org.nasdanika.sdk.runtime.models.meta.util;

    requires transitive org.nasdanika.sdk.runtime.models.core;
    requires transitive org.nasdanika.sdk.runtime.common;
    requires org.antlr.antlr4.runtime; // The text syntax's parser, not exported

    provides EPackage with MetaEPackageProvider;

    provides ResourceSetContributor with MetaResourceSetContributor;

}
```

It requires the runtime's core model and its `common` module transitively, which bring EMF and the OpenTelemetry API with them, and the ANTLR runtime, which parses the text syntax and is not part of the module's API. It provides two services: its `EPackage`, and a resource set contributor that registers the resource factory of the text syntax for the `nmeta` extension.

A resource set created with the runtime's `NasdanikaResourceSet.createAndConfigure()`, in the `common`

module, finds both on the module path and loads and saves vocabularies in every format the runtime supports: the text syntax, YAML and JSON with the runtime's `emf-json` module, XMI, and EMF's binary format. The listings of this book register what they use by hand, with EMF alone.

5.2. Names that clash

Meta's classes have the names of the concepts they model: `Package`, `Class`, `Enum`, `Object`. `Package`, `Class`, `Enum` and `Object` are also in `java.lang`, which every Java file imports. An explicit import of Meta's class wins over `java.lang` in that file, as in the listings of this book, and the JDK's class is then written with its package.

5.3. What the build does

The Xcore definition is in `src/main/resources/org/nasdanika/sdk/runtime/models/meta/meta.xcore`. Maven generates the Java sources into `src-gen` with the Xtext Maven plugin. The generated API is in three packages: the interfaces in `org.nasdanika.sdk.runtime.models.meta`, their implementations in `impl`, and the switch and adapter factory in `util`.

The grammar of the text syntax is in `src/main/grammar/Meta.g4`. The ANTLR Maven plugin generates its lexer and parser into `src-gen` as well, in the package `parser`, which is not exported: nothing outside the module sees a parse tree. The reader, the writer, the validator, the glossary, the resource and its factory are hand-written, in `resource`, and so are the package provider and the resource set contributor, in `services`.

Chapter 6. The model

Area	Classifiers
Base	`ModelElement`, on the core model's `NamedElement`, and `Annotation`
Structure	`Package`, `Classifier`, `Class`, `DataType`, `Enum`, `EnumLiteral`
Members	`TypedElement`, `Feature`, `Attribute`, `Reference`, `Operation`, `Parameter`
Types	`TypeParameter`, `TypeReference`, `ClassifierReference`, `TypeParameterReference`, `PrimitiveTypeReference`, `UnionTypeReference`, `IntersectionTypeReference`, `WildcardTypeReference`, `OpaqueTypeReference`, `PrimitiveKind`
Instances	`Object`, `IncomingReference`

The model is metacircular: a `Package` containing classes describes `Package` and `Class`, so a runtime can bootstrap from Meta alone and needs no foreign metamodel.

6.1. Base

Meta's `ModelElement` extends the core model's `NamedElement`, so every element has an `id`, a `name`, documentation as structure, markers, `uris` and an `icon` (Chapter 2). It contains `annotations`.

An `Annotation` has a `source`, a URI that names its owner, such as `urn:org.nasdanika` or the schema an element was loaded from, and `details`, a map of strings.

6.2. Structure

A `Package` is a namespace of classifiers, and the unit of publication: one package becomes one Maven artifact or one npm package. It has a `namespace` URI, its identity and what references resolve against, a short `prefix`, a `version`, and `classifiers`.

A `Classifier` is abstract and contains `typeParameters`, so that a data type may be parameterized too.

A `Class` is a type with features: what Ecore calls an `EClass`, what Java calls a class or an interface, and what JSON Schema calls an object schema. It has `abstract`, `superTypes` as type references, because a supertype may be parameterized, `features` and `operations`.

A `DataType` is a scalar type with a name of its own: `Instant`, `Duration`, `EmailAddress`. Its `representation` is how it crosses the wire and what a projection emits, an `Instant` is a string; its `sourceType`, such as `java.time.Instant`, is provenance only.

An `Enum` contains `literals`. An `EnumLiteral` has a serialized `value`, the name when unset, and an `ordinal`.

6.3. Members

A `TypedElement` has a `type`, a type reference, and a multiplicity: `lowerBound`, 0 for optional, and `upperBound`, 1 for single-valued and -1 for unbounded, with `ordered` and `unique`.

A `Feature` is state. `derived` means computed, not stored: a projection emits it as read-only or omits it. `readonly` is the positive form of Ecore's `changeable`, and `defaultValue` is verbatim, in the serialized form of the type.

An `Attribute` has `identifying`: it identifies its instance, and a reference may cite it. That is how a Domain-Driven Design entity is told from a value object.

A `Reference` has `containment`, which separates the edges of a tree from the edges of a graph, an `opposite` for the other end of a bidirectional reference, and `keys`: the attributes that identify a target within the containing list.

An `Operation` is a signature of behavior, carried for fidelity when loading from Java or Ecore and rendered as documentation. It has `typeParameters`, `parameters` and `exceptions`; a `Parameter` may be `variadic` and has a `defaultValue`.

6.4. Types

A type reference stands in a type position. The set is small on purpose: enough to write a declaration, with an escape hatch.

Class	What it references
`ClassifierReference`	A classifier: `target` within a model, or `namespace` and `classifierName` for one in a package the model does not contain, with `typeArguments`
`TypeParameterReference`	A type parameter, by `target` or `parameterName`

Class	What it references
`PrimitiveTypeReference`	A `PrimitiveKind`
`UnionTypeReference`	One of several `types`: JSON Schema's `oneOf`, an XSD choice, a TypeScript union
`IntersectionTypeReference`	All of several `types`; several bounds of a type parameter are one of these
`WildcardTypeReference`	`?`, `? extends T` or `? super T`, as a type argument only
`OpaqueTypeReference`	Verbatim `text` for what no class covers, such as conditional and mapped types, with the `references` it names, so that opaque text does not defeat import management

`PrimitiveKind` is a vocabulary of serialization, not of a language: `STRING`, `NUMBER`, `BOOLEAN`, `BIGINT`, `DATE`, `TIME`, `DATE_TIME`, `DURATION`, `BINARY`, `ANY`, `NULL` and `VOID`. `NUMBER` covers every numeric width, because JSON has one number and TypeScript has one number.

A projection without wildcards emits the bound that is set: the upper bound for `? extends T`, the lower bound for `? super T`, and the widest type it has for a bare `?`, `unknown` in TypeScript.

6.5. Instances

`Object` and `IncomingReference` are the reflective instances of Meta classes, the subject of Chapter 10.

Chapter 7. A vocabulary

The example of this book is the vocabulary of a library: people, books and the formats books are published in. It is built in Java, with three small builders.

7.1. Builders

Listing 2. Builders (`Vocabulary.java`)

```
static Class type(Package vocabulary, String name, String definition) {
    Class type = META.createClass();
    type.setName(name);
    type.setDocumentation(definition);                    ①
    vocabulary.getClassifiers().add(type);
    return type;
}

static Attribute attribute(
        Class owner,
        String name,
        PrimitiveKind kind,
        boolean identifying,
        String definition) {
    Attribute attribute = META.createAttribute();
    attribute.setName(name);
    attribute.setDocumentation(definition);
    attribute.setIdentifying(identifying);                ②
    attribute.setLowerBound(identifying ? 1 : 0);
    PrimitiveTypeReference type = META.createPrimitiveTypeReference();
    type.setKind(kind);
    attribute.setType(type);                              ③
    owner.getFeatures().add(attribute);
    return attribute;
}

static Reference reference(Class owner, String name, Class target, String definition) {
    Reference reference = META.createReference();
    reference.setName(name);
    reference.setDocumentation(definition);
    reference.setUpperBound(-1);                          ④
    ClassifierReference type = META.createClassifierReference();
    type.setTarget(target);
    reference.setType(type);
    owner.getFeatures().add(reference);
    return reference;
}
```

① Documentation is the core model's structure, not a string attribute of Meta.

② The attribute identifies its instance, as the identity of an entity in Domain-Driven Design.

③ The type is a reference to a primitive kind, which is how a value crosses the wire.

④ Many-valued.

7.2. The vocabulary

Listing 3. The library's vocabulary (`Vocabulary.java`)

```
public static Package library() {
    Package vocabulary = META.createPackage();
    vocabulary.setName("library");
    vocabulary.setNamespace("https://example.org/vocabulary/library");
    vocabulary.setVersion("2026.10.0");
    vocabulary.setDocumentation("The words a library uses, and what they mean here.");

    Class person = type(vocabulary, "Person",
            "Someone who wrote, edited or translated a book.");
    attribute(person, "name", PrimitiveKind.STRING, true,
            "The name as printed on the title page.");

    Enum format = META.createEnum();
    format.setName("Format");
    format.setDocumentation("How a book is published.");
    for (String name: new String[] { "HARDCOVER", "PAPERBACK", "EBOOK" }) {
        EnumLiteral literal = META.createEnumLiteral();
        literal.setName(name);
        literal.setOrdinal(format.getLiterals().size());
        format.getLiterals().add(literal);
    }
    vocabulary.getClassifiers().add(format);

    Class book = type(vocabulary, "Book",
            "A published work, identified by its ISBN.");
    attribute(book, "isbn", PrimitiveKind.STRING, true,
            "International Standard Book Number, 13 digits.");
    attribute(book, "title", PrimitiveKind.STRING, false,
            "The title as printed, subtitle excluded.");
    attribute(book, "published", PrimitiveKind.DATE, false,
            "The date of the first edition.");
    reference(book, "authors", person,
            "The people credited as authors, in the order printed.");

    Annotation source = META.createAnnotation();                    ①
    source.setSource("https://example.org/catalog/schema");
    source.getDetails().put("table", "BOOKS");
    book.getAnnotations().add(source);
    return vocabulary;
}
```

① What a source expressed and Meta does not: here, the table a harvester found the books in.

7.3. Saving and loading

A vocabulary is a model, so it is saved, versioned, compared and reviewed like any other:

Listing 4. Saving and loading a vocabulary (`MetaBookTests.java`)

```
URI uri = URI.createFileURI(directory.resolve("library.xml").toString());
Resource resource = resourceSet().createResource(uri);
resource.getContents().add(Vocabulary.library());
resource.save(null);                                              ①

Package loaded = (Package) resourceSet()
    .getResource(uri, true)
    .getContents()
    .getFirst();
assertThat(loaded.getClassifiers())
    .extracting(c -> c.getName())
    .containsExactly("Person", "Format", "Book");
```

① A file in a repository. A change to what a term means is a pull request whose diff shows which features moved.

The resource set of the listing registers the XMI factory and the Meta package by hand. In an application on the runtime, `NasdanikaResourceSet.createAndConfigure()` does both from the module path, registers the text syntax, and adds YAML and JSON when the runtime's `emf-json` module is there.

Building a vocabulary in Java shows the model, and it is how a harvester or a projection builds one. It is not how a person should write one: the next chapter writes a bank's vocabulary in the text syntax, and writes this library as text too.

Chapter 8. The text syntax

A vocabulary built in Java is a program that builds a vocabulary. A vocabulary written in Meta's text syntax is the vocabulary itself: a file a developer reads top to bottom, diffs in a pull request, and edits in any editor.

The syntax writes metamodels the way Xcore writes Ecore models. Files have the extension `.nmeta`, one file is one package, and they are parsed by the ANTLR runtime alone, with no Xtext and no Eclipse. The grammar, `src/main/grammar/Meta.g4`, documents every construct and how it maps to the model.

The example of this chapter is the vocabulary of a bank: the products it sells, its customers, their accounts, the transactions posted to them and the statements that report them. It is the vocabulary an assistant over a bank's systems needs before it can answer a question about any of them, and a business glossary the bank's people can read.

8.1. A package and its imports

A file starts with its package:

Listing 5. The package of the bank (`bank.nmeta`)

```
/**
 * The words a bank uses with its customers, and what they mean here.
 *
 * Kept by the business and reviewed as a pull request, like the code it is generated into.
 */
package bank namespace "https://example.org/vocabulary/bank" prefix "bank" version "2026.10.0"

import "https://example.org/vocabulary/party" as party
import "https://runtime.sdk.nasdanika.org/models/occupancy" as occupancy
annotation "urn:org.nasdanika" as Nasdanika
```

- The package has a name, and its namespace URI is its identity: what other packages import, and what a reference to one of its classes resolves against. The prefix and the version are optional.
- `import` names another package by its namespace, under an alias. `party.Party` is then the class `Party` of that package. A namespace is a name, not an address: nothing is fetched from it.
- `annotation` declares an alias for an annotation source, so that `@Nasdanika(icon = "bi-box")` is an

annotation with the source `urn:org.nasdanika` and a detail `icon`.

- A documentation comment, `/** … */`, is the Markdown documentation of the element that follows it. Other comments are ignored.

8.2. Data types and enumerations

Listing 6. Data types and enumerations (`bank.nmeta`)

```
/** An ISO 4217 currency code, such as USD or CAD. */
type CurrencyCode as string wraps java.util.Currency

/** An amount in the currency of its account, a decimal string so that no cent is lost. */
type Amount as string wraps java.math.BigDecimal

/** The kind of product, which decides what an account can do. */
enum ProductCategory {
    CHECKING,
    SAVINGS,
    CREDIT_CARD as "credit-card"
}

/** Whether a transaction takes money out of an account or puts it in. */
enum Direction {
    DEBIT,
    CREDIT
}
```

A data type has a name of its own and says how it crosses the wire: an `Amount` is a string, so that no projection turns money into a floating-point number. `wraps` records the type it stands for in a source language, as provenance. An enumeration's literals may have a serialized form, `as "credit-card"`, and an ordinal, `= 3`; without one, a literal's ordinal is one more than the previous literal's.

8.3. Classes

Listing 7. Products and customers (`bank.nmeta`)

```
/** What the bank sells: an offer with terms, under which accounts are opened. */
@Nasdanika(icon = "bi-box")
class Product {
    /** Identifies the product in the catalog. */
    id string[1] code

    /** The name customers see, such as "Everyday Checking". */
    string[1] name
```

```
    /** What kind of product it is. */
    ProductCategory[1] category

    /** The currencies accounts of this product can be held in. */
    CurrencyCode[+] currencies
}
/** A party the bank has a relationship with, through at least one account. */
@Nasdanika(icon = "bi-person")
class Customer {
    /** Identifies the customer, and is printed on statements. */
    id string[1] customerNumber

    /** Who the customer is. Defined by the party vocabulary, not here. */
    refers party.Party[1] party

    /** When the relationship started. */
    date since

    /** The accounts the customer holds, alone or jointly. */
    refers Account[*] accounts opposite holders

    /** In Quebec, consumer contracts are in French unless the customer asks otherwise. */
    @occupancy.Presence { jurisdiction: "CA-QC" }
    string contractLanguage = "fr"
}
```

Listing 8. Accounts (`bank.nmeta`)

```
/** An arrangement under a product, in one currency, through which money moves. */
@Nasdanika(icon = "bi-bank")
class Account {
    /** Identifies the account, and is printed on cheques and statements. */
    id string[1] accountNumber

    /** The product the account was opened under, which sets its terms. */
    refers Product[1] product

    /** The customers who own the account: one, or several for a joint account. */
    refers Customer[+] holders opposite accounts

    /** The currency the account is held in, one of its product's. */
    CurrencyCode[1] currency

    /** Every transaction posted to the account, addressed by its identifier. */
    contains Transaction[*] transactions keys transactionId

    /** The statements issued for the account, oldest first. */
    contains Statement[*] statements

    /** The sum of posted transactions, computed rather than stored. */
    derived readonly Amount balance
```

```
    /** The balance at the end of a day, from the transactions posted until then. */
    op Amount balanceAt(date day)
}
```

A feature is a type followed by a name, as in Xcore:

- With `refers` or `contains` it is a reference, and `contains` makes the target part of this object: a statement belongs to its account, a product does not.
- Otherwise it is an attribute, of a primitive kind, such as `string`, `date` and `dateTime`, or of a data type or enumeration.
- `id` marks an attribute that identifies its instance, as the identity of an entity in Domain-Driven Design.
- `opposite` pairs two references, `keys` names the attributes that identify a contained object within its list, and `= "fr"` is a default value.
- `derived` and `readonly` say that a feature is computed rather than stored, and `op` declares the signature of an operation. Meta carries no bodies.

The multiplicity follows the type:

Written	Bounds	Meaning
`string name`	0..1	Optional, single-valued
`string[1] name`	1..1	Required
`string[?] name`	0..1	Optional, said explicitly
`Account[*] accounts`, `Account[] accounts`	0..*	Any number
`Customer[+] holders`	1..*	At least one
`string[2..5] names`, `string[2..*] names`	2..5, 2..*	A range

Types also combine. `string | number` is a union, `A & B` an intersection, `Repository<Account, string>` a class with type arguments, `? extends T` a wildcard, and text in backticks an opaque type, for what nothing else covers, such as TypeScript's `Record<string, unknown>`. They are rarely needed in a business vocabulary, and they are what lets a Java or TypeScript signature be written down without loss.

8.4. Loading

The resource factory of the text syntax is `MetaResourceFactory`. On the module path, the module's resource set contributor registers it for `.nmeta` files; this listing registers it by hand:

Listing 9. A resource set for vocabularies (`TextSyntaxTests.java`)

```
static ResourceSet resourceSet() {
    ResourceSet resourceSet = new ResourceSetImpl();
    Map<String, Object> factories = resourceSet.getResourceFactoryRegistry()
        .getExtensionToFactoryMap();
    factories.put("nmeta", new MetaResourceFactory());                  ①
    factories.put("xml", new XMIResourceFactoryImpl());
    resourceSet.getPackageRegistry().put(MetaPackage.eNS_URI, MetaPackage.eINSTANCE);

    Path party = BOOK.resolve("party.nmeta").toAbsolutePath();
    resourceSet.getURIConverter().getURIMap().put(                      ②
            URI.createURI("https://example.org/vocabulary/party"),
            URI.createFileURI(party.toString()));
    return resourceSet;
}
```

① The text syntax.

② The party vocabulary is owned by another team. The URI map says where its namespace is found here: a file next to this one, in a test.

A vocabulary then loads like any other model:

Listing 10. Loading the bank's vocabulary (`TextSyntaxTests.java`)

```
Resource resource = resourceSet().getResource(BANK, true);
Package bank = (Package) resource.getContents().getFirst();
assertThat(bank.getClassifiers()).extracting(Classifier::getName).containsExactly(
        "CurrencyCode", "Amount", "ProductCategory", "Direction",
        "Product", "Customer", "Account", "Transaction", "Statement");

Class customer = (Class) classifier(bank, "Customer");
ClassifierReference party = (ClassifierReference) feature(customer, "party").getType();
assertThat(party.getTarget().getName()).isEqualTo("Party");                 ①
assertThat(party.getTarget().eResource().getURI().lastSegment()).isEqualTo("party.nmeta");

Class account = (Class) classifier(bank, "Account");
Reference holders = (Reference) feature(account, "holders");
assertThat(holders.getOpposite()).isSameAs(feature(customer, "accounts"));
assertThat(holders.getLowerBound()).isEqualTo(1);                           ②
assertThat(holders.getUpperBound()).isEqualTo(-1);
assertThat(account.getDocumentation())
    .isEqualTo("An arrangement under a product, in one currency, through which money moves.");
```

```
assertThat(resource.getWarnings()).extracting(Resource.Diagnostic::getMessage)  ③
    .containsExactly("Facet type occupancy.Presence is not available, "
            + "its properties are not checked");
```

① `party.Party` resolves to a class of another resource, loaded on demand from where the URI map says.

② `Customer[+] holders`: at least one.

③ The occupancy facet's type is not available. That is a warning, not an error (Section 8.6).

8.5. Federation

The party vocabulary is a file of its own, and the bank's vocabulary does not care where it comes from:

Listing 11. The party vocabulary (`party.nmeta`)

```
/**
 * The people and organizations the bank does business with.
 * Owned by the party team and used by every other vocabulary of the bank.
 */
package party namespace "https://example.org/vocabulary/party" version "2026.10.0"

/** A person or an organization: whoever can hold an account or sign a contract. */
abstract class Party {
    /** Identifies the party in every system of the bank. */
    id string[1] partyId

    /** The name as on the party's identity documents or registration. */
    string[1] legalName
}

/** A natural person. */
class Person extends Party {
    date dateOfBirth
}

/** A company, a partnership, a trust or another legal entity. */
class Organization extends Party {
    /** The number the entity is registered under in its jurisdiction. */
    string registrationNumber
}
```

A type name is resolved against the type parameters in scope, then against the classifiers of the file, wherever in it they are declared. A name qualified with an alias, `party.Party`, becomes a classifier reference with the imported namespace and the class name. Its target is set when the

package of that namespace is available: loaded in the resource set already, or loadable because the URI converter maps the namespace to a location. Otherwise the reference keeps the namespace and the name, and the package is read on its own, which is what lets a team publish its vocabulary without the vocabularies it refers to.

8.6. Facets

An annotation is a bag of strings keyed by its source. A facet is typed: `@occupancy.Presence { jurisdiction: "CA-QC" }` says that the contract language of a customer applies in Quebec, with a class of the occupancy model as the type of what it says. Its value is an object literal, with strings, numbers, lists in `[ ]`, nested objects and names of enumeration literals and elements as values.

A facet is checked against its type when the type is available: a property its class has no feature for is a warning. When the type is not available, as here, the facet is read without checking, and the warning says so.

Until the core model has facets, a facet is recorded as an annotation **SKETCH**: its source is the URI of its type, the namespace and the class name joined with `#`, its name is the class name, and its properties are details. An object value is a nested annotation named after its property, and a list one whose source is `MetaReader.LIST`, with its items keyed by position.

Listing 12. A facet, as recorded (`TextSyntaxTests.java`)

```
Class customer = (Class) classifier(bank, "Customer");
Annotation presence = feature(customer, "contractLanguage").getAnnotations().getFirst();
assertThat(presence.getSource())
    .isEqualTo("https://runtime.sdk.nasdanika.org/models/occupancy#Presence");
assertThat(presence.getDetails().get("jurisdiction")).isEqualTo("CA-QC");
```

Roles, owners and occupancy are what facets are for: data about an element that another model defines and validates (Section 13.2).

8.7. Problems and provenance

The reader collects what is wrong in the resource's `getErrors()` and `getWarnings()`, each with its file, line and column, and loading fails with the first error, as EMF's XML resources do:

- A syntax error leaves the resource empty.

- An unknown type, an unknown alias, or an opposite or a key that does not exist is an error, and the resource keeps what could be read, so that a caller catching the exception still sees it.
- A facet whose type is not available, or a property its type does not have, is a warning.

Every package, classifier, feature, operation, parameter and enum literal read from a file gets a core model marker with the file's URI and the `line:column` where its declaration starts. When a generated page or a projected schema says something surprising, the marker says which line of which file it came from.

8.8. Validating a directory

A resource reports the problems of one file, and a vocabulary is usually several. `MetaValidator` reads every `.nmeta` file under a directory, as a build does:

Listing 13. Validating the vocabularies of a directory (`TextSyntaxTests.java`)

```
List<Problem> problems = MetaValidator.validate(BOOK, BOOK);                ①
assertThat(problems).extracting(Problem::toString).containsExactly(         ②
        "bank.nmeta:70:6: warning: Facet type occupancy.Presence is not available, "
        + "its properties are not checked");
assertThat(MetaValidator.hasErrors(problems)).isFalse();                    ③
```

① The directory to search, recursively, and the directory the reported file names are relative to.

② `bank.nmeta` imports `party.nmeta`, and no URI map is needed: the files are read together.

③ Warnings do not fail a build. What does is up to the caller, and this is what it asks.

The files are read twice. The first pass learns the namespace of each package, and the second loads them all in one resource set whose URI map points each namespace to its file, so an import resolves whichever file is read first. That decides what the validator can know:

- A type the imported package does not have is an error, when that package is in the directory: `party.Partyy` is `Unknown type Partyy in example.org/party`.
- An import of a namespace no file in the directory has is not checked, which is what lets a team validate its vocabulary without the vocabularies of other teams.
- Two files with one namespace are an error, since a namespace identifies one package.

Problems come errors first, then by file and position, one per line in the format of compilers:

Listing 14. What a typo reports (`TextSyntaxTests.java`)

```
String typo = Files.readString(BOOK.resolve("bank.nmeta"))
    .replace("refers Account[*] accounts", "refers Acount[*] accounts");
Files.writeString(meta.resolve("bank.nmeta"), typo);

List<Problem> problems = MetaValidator.validate(meta, directory);
assertThat(problems).extracting(Problem::toString).containsExactly(
        "src/main/meta/bank.nmeta:67:12: error: Unknown type Acount",       ①
        "src/main/meta/bank.nmeta:67:40: error: The opposite of accounts "
        + "is in a class that is not available: holders",                   ②
        "src/main/meta/bank.nmeta:70:6: warning: Facet type occupancy.Presence "
        + "is not available, its properties are not checked");
```

① The file, the line and the column of the problem.

② A problem can have consequences, reported with it. Fixing the first error fixes this one too.

That format is the point. A person can read it, VS Code's problem matchers parse it, and an assistant can read it and go to the line, which is what lets an assistant correct its own drafts (Chapter 11).

8.9. Saving

A vocabulary read from text is a model, so it is saved in any format a resource set supports. References into a `.nmeta` file are by name, not by position, so they survive reordering the file:

Listing 15. Saving as XMI (`TextSyntaxTests.java`)

```
ResourceSet resourceSet = resourceSet();
Package bank = (Package) resourceSet.getResource(BANK, true).getContents().getFirst();
URI xmi = URI.createFileURI(directory.resolve("bank.xml").toString());
Resource copy = resourceSet.createResource(xmi);
copy.getContents().add(bank);                                          ①
copy.save(null);
assertThat(Files.readString(directory.resolve("bank.xml")))
    .contains("party.nmeta#//Party");                                  ②
```

① The package moves to an XMI resource. Its references to the party vocabulary stay where they were.

② A reference to the class `Party` of `party.nmeta`.

The other direction is `MetaWriter`, which writes a package as text, and which the resource uses to save a `.nmeta` file. Here is the library of the previous chapter, built in Java:

Listing 16. Writing a vocabulary as text (`TextSyntaxTests.java`)

```
Package library = Vocabulary.library();
String text = MetaWriter.write(library);                                    ①

URI uri = URI.createFileURI(directory.resolve("library.nmeta").toString());
Resource resource = resourceSet().createResource(uri);
resource.getContents().add(library);
resource.save(null);                                                        ②
```

① The text, as a string.

② The same text, as a file.

Listing 17. The library's vocabulary as text (`library.nmeta`)

```
/** The words a library uses, and what they mean here. */
package library namespace "https://example.org/vocabulary/library" version "2026.10.0"

annotation "https://example.org/catalog/schema" as Schema

/** Someone who wrote, edited or translated a book. */
class Person {
    /** The name as printed on the title page. */
    id string[1] name
}

/** How a book is published. */
enum Format {
    HARDCOVER,
    PAPERBACK,
    EBOOK
}

/** A published work, identified by its ISBN. */
@Schema(table = "BOOKS")
class Book {
    /** International Standard Book Number, 13 digits. */
    id string[1] isbn

    /** The title as printed, subtitle excluded. */
    string title

    /** The date of the first edition. */
    date published

    /** The people credited as authors, in the order printed. */
    refers Person[*] authors
}
```

The text is canonical, not a copy of a source: comments other than documentation and the

layout of the source are not kept, and the imports and annotation aliases are derived from the namespaces and sources the package uses. Reading what was written and writing it again gives the same text, which the tests check. That makes the text syntax a target as well as a source: a vocabulary harvested from a database or authored in a spreadsheet can be written as text, reviewed as text, and compared with last month’s as text.

8.10. The bank’s glossary

The glossary projection of the next chapter works on any package, wherever it came from. The bank’s accounts, as the business reads them:

Listing 18. The accounts of the bank’s glossary (`bank.md`)

```
## Account

An arrangement under a product, in one currency, through which money moves.

| Term | Type | Rule | Meaning |
|---|---|---|---|
| accountNumber | text | required, identifies it | Identifies the account, and is printed on
cheques and statements. |
| product | Product | required | The product the account was opened under, which sets its terms.
|
| holders | Customer | at least one, the other side of Customer.accounts | The customers who own
the account: one, or several for a joint account. |
| currency | CurrencyCode | required | The currency the account is held in, one of its product's.
|
| transactions | Transaction | any number, part of it | Every transaction posted to the account,
addressed by its identifier. |
| statements | Statement | any number, part of it | The statements issued for the account, oldest
first. |
| balance | Amount | computed | The sum of posted transactions, computed rather than stored. |

What it can do:

- **balanceAt**(day): The balance at the end of a day, from the transactions posted until then.
```

The file is generated from `bank.nmeta`, and the tests compare it with a reviewed copy.

Chapter 9. Projections

A vocabulary is worth writing down once because of what is projected from it. The runtime has the first projection below, a glossary. Its other projections are PLANNED, and the tests of this book sketch one of them, to Ecore, to show the idea.

9.1. A glossary, for people

The first projection is for people: a glossary that cannot drift from the types, because both come from one source.

Listing 19. A glossary (`MetaBookTests.java`)

```
String glossary = MetaGlossary.write(Vocabulary.library());                ①

List<Path> written = MetaGlossary.generate(                                ②
        Path.of("src/main/skill/nasdanika-meta/examples"),
        directory.resolve("glossary"));
```

① One package, as Markdown.

② A glossary for each `.nmeta` file under a directory, read together as the validator reads them (Section 8.8), and returned as the list of files written. Validate first: a file with an error fails here.

A glossary is for people who do not read the notation, so `MetaGlossary` does not use it. The Rule column states what the structure says, and the types are in plain words: a `string` is text, a `boolean` is yes or no, and a class from another package says so.

In the file	In the Rule column
No multiplicity, `[1]`, `[*]`, `[+]`	optional, required, any number, at least one
`[3]`, `[2..5]`, `[0..5]`, `[2..*]`	exactly 3, 2 to 5, up to 5, at least 2
`id`	identifies it
`contains`	part of it
`opposite holders` on `Customer.accounts`	the other side of Account.holders
`derived`	computed
`readonly`	read-only

In the file	In the Rule column
`= "fr"`	fr by default

Annotations and facets are left out.

For the library's vocabulary, it writes this Markdown, which the test compares with a reviewed file:

Listing 20. The glossary of the library

```
# library

Version 2026.10.0.

The words a library uses, and what they mean here.

## Person

Someone who wrote, edited or translated a book.

| Term | Type | Rule | Meaning |
|---|---|---|---|
| name | text | required, identifies it | The name as printed on the title page. |

## Format

How a book is published.

One of:

- **HARDCOVER**
- **PAPERBACK**
- **EBOOK**

## Book

A published work, identified by its ISBN.

| Term | Type | Rule | Meaning |
|---|---|---|---|
| isbn | text | required, identifies it | International Standard Book Number, 13 digits. |
| title | text | optional | The title as printed, subtitle excluded. |
| published | date | optional | The date of the first edition. |
| authors | Person | any number | The people credited as authors, in the order printed. |
```

9.2. A dynamic Ecore package, for tools

The second projection is for tools: a dynamic Ecore package, so that every EMF tool works on a vocabulary without generated Java.

Listing 21. An Ecore projection (Projections.java)

```
/**
 * A sketch of the projection to Ecore: classes, attributes and references, enough for EMF to
 * create, validate and serialize instances without generated code.
 */
public static EPackage ecore(Package vocabulary) {
    EcoreFactory ecore = EcoreFactory.eINSTANCE;
    EPackage ePackage = ecore.createEPackage();
    ePackage.setName(vocabulary.getName());
    ePackage.setNsURI(vocabulary.getNamespace());
    ePackage.setNsPrefix(vocabulary.getName());

    Map<Class, EClass> classes = new HashMap<>();
    for (Classifier classifier: vocabulary.getClassifiers()) {
        if (classifier instanceof Class type) {
            EClass eClass = ecore.createEClass();
            eClass.setName(type.getName());
            ePackage.getEClassifiers().add(eClass);
            classes.put(type, eClass);
        }
    }
    classes.forEach((type, eClass) -> {
        for (Feature feature: type.getFeatures()) {
            if (feature instanceof Attribute attribute
                    && attribute.getType() instanceof PrimitiveTypeReference primitive) {
                EAttribute eAttribute = ecore.createEAttribute();
                eAttribute.setName(attribute.getName());
                eAttribute.setEType(dataType(primitive.getKind()));
                eAttribute.setLowerBound(attribute.getLowerBound());
                eAttribute.setUpperBound(attribute.getUpperBound());
                eAttribute.setID(attribute.isIdentifying());
                eClass.getEStructuralFeatures().add(eAttribute);
            } else if (feature instanceof Reference reference
                    && reference.getType() instanceof ClassifierReference target) {
                EReference eReference = ecore.createEReference();
                eReference.setName(reference.getName());
                eReference.setEType(classes.get((Class) target.getTarget()));
                eReference.setContainment(reference.isContainment());
                eReference.setLowerBound(reference.getLowerBound());
                eReference.setUpperBound(reference.getUpperBound());
                eClass.getEStructuralFeatures().add(eReference);
            }
        }
    });
    new ResourceImpl(URI.createURI(ePackage.getNsURI())).getContents().add(ePackage);
    return ePackage;
```

```
}

private static EDataType dataType(PrimitiveKind kind) {
    return switch (kind) {
        case NUMBER -> EcorePackage.Literals.EDOUBLE;
        case BOOLEAN -> EcorePackage.Literals.EBOOLEAN;
        case DATE, DATE_TIME -> EcorePackage.Literals.EDATE;
        default -> EcorePackage.Literals.ESTRING;
    };
}
```

EMF then creates, validates and saves instances of the vocabulary's classes:

Listing 22. Instances of a projected vocabulary (`MetaBookTests.java`)

```
EPackage library = Projections.ecore(Vocabulary.library());
EClass bookClass = (EClass) library.getEClassifier("Book");
EObject book = EcoreUtil.create(bookClass);                    ①
book.eSet(bookClass.getEStructuralFeature("isbn"), "978-0321125217");
book.eSet(bookClass.getEStructuralFeature("title"), "Domain-Driven Design");

ResourceSet resourceSet = resourceSet();
resourceSet.getPackageRegistry().put(library.getNsURI(), library); ②
URI uri = URI.createFileURI(directory.resolve("books.xml").toString());
Resource resource = resourceSet.createResource(uri);
resource.getContents().add(book);
resource.save(null);
```

① An instance of a class that has no Java class.

② The package is registered with the resource set, so its instances can be saved and loaded.

The identifying attribute becomes the instance's EMF ID, so references to a book in a file are by its ISBN.

The Ecore projection is the quiet one and possibly the most useful: a vocabulary captured in a spreadsheet can be validated, compared, diagrammed and serialized by tools that already exist. It is also the conformance test of Meta, since a vocabulary projected to Ecore and back must come back the same, and the production path for models written in Meta that need Java.

9.3. What a projection decides

A projection decides what the neutral model leaves open:

- whether a `Class` becomes an interface, a type alias or a class;

- how a `DataType` is represented, from its `representation`;
- what to do with what the target cannot say: wildcards, unions in Java, operations in a data schema;
- which annotations it reads, by their source.

The model stays the same for every projection, which is what lets one vocabulary serve many.

Chapter 10. Instances whose type is a reference

`Object` is the reflective instance of a Meta class: no generated interface, no implementation class, no factory. It knows its type by reference, and answers about itself through a dozen operations:

Operation	What it answers or does
`getIdentities()`	Its identities: none for a value object, one or more for an entity
`getContainer()`, `getContainmentFeature()`	Where it is contained
`get(feature)`, `get(feature, from, count)`	A value, or a range of a many-valued feature without the rest
`set`, `add`, `remove`, `unset`	Changes. `unset` restores the unset state, which is not the same as setting a default
`size(feature)`	How many values a feature holds: zero for unset
`invoke(operation, arguments)`	Calls an operation, implemented elsewhere
`getIncomingReferences(references)`, with a range	Who references it, through which references, at which positions
`getIncomingReferenceCount(references)`	How many, or -1 when it cannot be known here
`isResolved()`	False when it stands for an object that has not been loaded

An agent gets an interaction surface that can be taught in a paragraph, not a package.

10.1. The rule that keeps it small

An operation belongs to `Object` only if a consumer cannot compute it from the others and the type without loading data it did not ask for. Contents are derivable from the type's containment references and `get`, so there is no operation for contents. `size` and the ranged `get` pass the rule, because they answer without fetching every value, and the ranged `get` is what makes a large model usable over a network. `isResolved` is the one concession: references cross resources by URI, and something has to represent "not here yet".

10.2. Classified later

Because the type is a reference, an object can exist before it is classified, and be classified later. That is the shape elicitation has: a thing is named before anyone knows what kind of thing it is, refinement proceeds by dividing, an agent proposes a classification and a person corrects it. Before classification, values wait in annotations; classification promotes what fits into features and leaves the rest where it was.

Change	Example	Safety
Specialization, to a subtype	`Person` to `Author`	Safe: every feature in scope stays in scope
Generalization, to a supertype	`Author` to `Person`	Loses a distinction
Reclassification, to an unrelated type	`Person` to `Publisher`	Arbitrary: names may coincide while meanings do not

Values are never silently dropped. A value whose feature falls out of scope moves to an annotation, keeping its feature name and the type that declared it, so a wrong classification can be undone. Identity does not come from an identifying attribute, so it survives retyping.

10.3. What it makes possible

- A vocabulary as a model whose terms are classes and whose permitted values are instances, serving a generated glossary for people and a lookup tool for an agent from one source.
- A user interface over anything Meta describes, with no generated code.
- Queries over instances and their types in one statement, because the type is an edge like any other.

10.4. Status

The operations of `Object` are declared and have no implementation **SKETCH**. The obvious implementation is dynamic EMF behind the interface, through the Ecore projection of Chapter 9: EMF provides resources, serialization, change recording and proxy resolution, and callers see a dozen operations instead of thirty.



The comments on `Object` in the Xcore speak of an intrinsic `id`, of "six operations", and of a marker recording who classified an object. `Object` has no `id` feature, but `getIdentities()`; it has more than a dozen operations; and it extends nothing, so it has no markers. Decide whether `Object` should extend the core model's `Marked`, or `ModelElement`, and update the comments.

Chapter 11. AI-assisted domain design

The people who know what the words of a domain mean are rarely the people who write them in a precise notation. A product manager at a bank can spot a wrong definition of a joint account in a second, and has no reason to learn a grammar. The formats of Chapter 12 will meet such people in the spreadsheets and diagrams they already use **PLANNED**. An AI assistant can meet them today: they talk, or hand over what they have, the assistant writes the text syntax, the build checks it, and they correct what it says about their business.

This chapter is written first for those subject-matter experts, and needs no code as far as Section 11.5. From there on, it is for the person who sets the project up.

11.1. Who checks what

An assistant is good at the mechanical part of modeling: turning a description, a glossary, a spreadsheet extract or meeting notes into structure. Correcting a draft is much cheaper than writing one from nothing. But an assistant writes a plausible definition of a term it has never heard with the same ease as a correct one, and a plausible definition can get past a reader who does not know the domain. So review only works where the reviewer can judge what is reviewed, and this way of working gives each check to whoever can:

Who	Judges	Cannot judge
The assistant	Nothing final. It drafts, and it fixes what the validator reports	What the words mean here, unless someone told it
The validator, in the build	Whether the file is well formed: the syntax, that every type exists, that opposites and keys exist, that imports within the project resolve	Whether a definition is right, or a rule is true
The subject-matter expert	Whether the definitions and the rules are true of the business	Whether the file is well formed, and does not have to
A developer or an architect	How the vocabulary fits with others: its namespace, what other teams already own, whether a change is safe to publish	What the business means by its words

The structure and the meaning are the two things a vocabulary can get wrong, and they fail in different hands. A vocabulary the validator accepts can still be wrong about the business, and it is the expert who decides that it is not.

11.2. How it goes

1. **Bring what exists.** A glossary, a spreadsheet extract, a policy document, meeting notes, or your own words. An assistant works from what it is given, so the more of your organization's own language it sees, the less it invents.
2. **Answer what it cannot infer.** It asks what the vocabulary's namespace is, a name that identifies it and belongs to your organization, and which terms other teams already own.
3. **It drafts.** One file per package, with a definition for every term, written for a business reader.
4. **The build checks the draft.** The assistant runs the validator, reads what it reports, and fixes every error at its line and column. You do not have to look at any of this. It tells you about warnings rather than hiding them.
5. **It tells you what it did**, in terms of the business: the terms added, renamed or redefined, and anything it was unsure of.
6. **It reads the rules back**, as plain statements without the notation, the ones it guessed first, and points you to a glossary the build has written.
7. **You correct it, in your own words.** "An account can't be without a holder." "We don't say client, we say customer." It changes the file, and the build checks it again.
8. **The change is reviewed like any other.** The vocabulary is a file in Git, so the change is a pull request: a developer or an architect looks after the structure, and you approve the meaning.

11.3. Reading a vocabulary without learning the notation

The definitions are prose, and you can read them as they are. The rules are in the structure, where they are easy to miss, and they are often the part an assistant guessed, because notes and glossaries rarely say whether an account can have no holder. You get them in two ways, and neither asks you to read the notation.

The first is the conversation. The skill has the assistant read the rules back after every draft or change, and each statement is a question only you can answer. From the bank's vocabulary (Chapter 8):

In the file	What to ask yourself
`refers Customer[+] holders`	The file says an account has at least one holder. Can an account ever have none, say after its only holder dies?
`contains Statement[*] statements`	The file says a statement is part of its account, and does not exist without it. Do you have to keep statements after an account is closed?
`refers Product[1] product`	The file says an account has exactly one product, which has a life of its own. Can an account move to another product?
`CurrencyCode[1] currency`	The file says an account is in exactly one currency. Is there a multi-currency account?
`id string[1] accountNumber`	The file says the number identifies the account. Is a number ever reused?

A wrong answer here is a correction to one line, made today. The same wrong answer found in a generated schema, a year from now, is a migration.

An assistant without the skill can be asked for the same thing:

```
Read this vocabulary back to me as plain statements of business rules, one for
each relationship and each required attribute. Mark the ones you guessed, and
ask me about those first.
```

The second is the glossary, which is how to check a whole vocabulary rather than the part that just changed. Each build without errors writes one, and since a program writes it from the file, it does not depend on what an assistant remembered to say. The Rule column states the structure in plain words:

Listing 23. The customers of the bank's glossary (`bank.md`)

```
## Customer

A party the bank has a relationship with, through at least one account.

| Term | Type | Rule | Meaning |
|---|---|---|---|
| customerNumber | text | required, identifies it | Identifies the customer, and is printed on
statements. |
| party | Party (from party) | required | Who the customer is. Defined by the party vocabulary,
not here. |
| since | date | optional | When the relationship started. |
```

```
| accounts | Account | any number, the other side of Account.holders | The accounts the customer
holds, alone or jointly. |
| contractLanguage | text | optional, fr by default | In Quebec, consumer contracts are in French
unless the customer asks otherwise. |
```

Read the Rule column against the Meaning column, and against what you know. Here they disagree: a customer is defined as having a relationship "through at least one account", while the rule for `accounts` says any number. The validator accepts this, and I left it in the bank's vocabulary because it is the kind of thing only you can settle. Either a customer can exist before their first account, and the definition is wrong, or they cannot, and the rule is.

11.4. What to watch for

The validator accepts every one of these, which is why they are yours to catch:

- **A definition that could be about anyone's business.** "The account of the customer" says nothing. Ask whether someone in your organization could disagree with it. If not, it is not yet a definition.
- **A rule stated more firmly than your sources do.** The notes said "usually one"; the file says exactly one.
- **One word for two things, or two words for one.** The sources say client and customer. An assistant may keep both, or merge them, without asking, and either could be wrong. Whichever you decide, it belongs in the definitions.
- **A term another team owns, copied instead of imported.** Copies drift. A developer or an architect usually spots this one in review.

What you give an assistant goes wherever that assistant sends it, so check what your organization allows before you paste anything in. A vocabulary needs definitions, not records: it describes what a customer is, and no customer's data has to be part of the conversation.

11.5. The agent skill

An agent skill is a directory of instructions that an assistant loads when a task calls for them: a `SKILL.md` with a name and a description of when to use it, and whatever files it refers to. The skill `nasdanika-meta` teaches an assistant the workflow above, including reading the rules back, the text syntax and the conventions of a vocabulary. It is in the runtime's repository, not in the model's

jar, at `models/meta/model/src/main/skill/nasdanika-meta`:

File	What it is
`SKILL.md`	The workflow, how to read the rules back, the conventions, the syntax at a glance, and the pitfalls
`references/syntax.md`	The full syntax, read when it is needed
`examples/`	`bank.nmeta`, `party.nmeta` and `library.nmeta`, the files of this book
`assets/vocabulary-project/`	The starter project below

To install it, copy the directory to where your assistant finds skills. For Claude Code, that is `.claude/skills/nasdanika-meta` in a project, or `~/.claude/skills/nasdanika-meta` for all of them. Everything in it is plain text, so an assistant that does not have skills can be pointed at the files.

The conventions are the part a language reference would not give an assistant:

- Every class, data type, enumeration and feature has a definition for a business reader, starting with one sentence that could stand alone in a glossary. Not how it is stored, and not "the X of the Y".
- The multiplicity says the business rule, `contains` means the target does not exist without its owner, and two-way relationships are paired.
- Money and identifiers are data types, not numbers, so that no projection turns money into a floating-point number.
- A term another team owns is imported by namespace, not copied, and a namespace is never invented under someone else's domain.

The model's tests, `SkillTests`, check what an assistant learns from the skill: its examples are the files of this book, they validate, every `nmeta` block in its Markdown loads without errors, its read-back examples are lines of `bank.nmeta`, and the glossary is where it says. They do not test an assistant. What it writes is checked by the validator and by the people who own the words.

11.6. A vocabulary project

The skill's starter project is what an assistant copies when there is no project yet, and it asks you where. Someone technical sets it up once: it needs a JDK for Java 25, Maven and the runtime's

artifacts, and its `pom.xml` needs the vocabulary's own coordinates.

```
vocabulary/
  pom.xml                                    the runtime's BOM, and Meta and JUnit for tests
  AGENTS.md, CLAUDE.md                       what an assistant must know about the project
  .gitignore                                 leaves out target
  .vscode/tasks.json                         a "Validate vocabulary" task
  src/main/meta/                             the vocabulary, one package per .nmeta file
  src/test/java/vocabulary/VocabularyTest.java
  target/glossary/                           written by the build, one .md per .nmeta file
```

The whole build is one test, which runs the validator of Section 8.8 and, when there are no errors, the glossary of Chapter 9:

Listing 24. The starter project's test (`VocabularyTest.java`)

```
package vocabulary;

import static org.junit.jupiter.api.Assertions.assertFalse;

import java.nio.file.Path;
import java.util.List;

import org.junit.jupiter.api.Test;
import org.nasdanika.sdk.runtime.models.meta.resource.MetaGlossary;
import org.nasdanika.sdk.runtime.models.meta.resource.MetaValidator;

/**
 * Validates the vocabulary in src/main/meta. Problems are printed one per line,
 * file:line:column: severity: message, which assistants read and the VS Code task's problem
 * matcher shows in the editor. A vocabulary without errors gets a glossary for the people who
 * own its words, in target/glossary.
 */
class VocabularyTest {

    @Test
    void validate() {
        List<MetaValidator.Problem> problems =
                MetaValidator.validate(Path.of("src/main/meta"), Path.of(""));
        problems.forEach(System.out::println);
        assertFalse(MetaValidator.hasErrors(problems), "The vocabulary has errors, listed
above");

        MetaGlossary.generate(Path.of("src/main/meta"), Path.of("target/glossary"))
            .forEach(glossary -> System.out.println("Glossary: " + glossary));
    }

}
```

`mvn -q test` prints the problems one per line and fails on an error, which is all an assistant needs to correct its own drafts. Run in the build of the repository, the same command stops a broken vocabulary from being merged. In VS Code, the task's problem matcher puts the same problems in the editor, and the Markdown preview shows the glossary.

The glossary is in `target`, so it is not committed, and a pull request shows the change to the `.nmeta` file and not to the glossary. To give an expert the glossary of a pull request, publish `target/glossary` from the build of the repository, in whatever way the team publishes such files.

Not everyone will have a build on their laptop. With no Maven, the skill has an assistant follow the examples, say that the file has not been validated and that there is no glossary, and read the rules back all the same. An expert can then work in a conversation with an assistant and hand the file over, and the build of the pull request does the checking.

Part III: Looking Ahead

The formats vocabularies are authored and published in, who each is for, flattening, variability and occupancy.

Chapter 12. Formats

A vocabulary is one model, and formats are how people put it in and get it out. The text syntax is the first format of this edition. The others below have been proven in Nasdanika's work outside the SDK, and will be migrated to the runtime as vocabularies need them PLANNED.

The rule that makes many formats affordable: every format reads into Meta and every output is projected from Meta. Nothing converts from one format to another directly, so a new input format works with every output there is, and a new output with every input.

12.1. Authoring formats

Format	Strengths	Weaknesses
Meta text, `.nmeta` (Chapter 8)	Compact and precise: generics, unions and opposites have a notation. Diffs and merges well, so a change to a term is a reviewable pull request. Written back by `MetaWriter`, so it round-trips	A language to learn, if a small one. No picture: relationships are read, not seen
Excel (Excel model) PLANNED	Where vocabularies already live, and what everybody can edit. Sorting, filtering and bulk changes for free, and a row per attribute suits classes with many of them	A binary file: diffs and merges are poor. Structure by convention, so references, multiplicities and types need columns and checks. Conventions drift between sheets
Draw.io, a visual schema (demo) PLANNED	Classes as boxes and relationships as connections, so the shape of the domain is seen. The diagram is the source, not a picture of it, and it is what a workshop draws anyway	Attributes clutter boxes. Types and multiplicities are awkward to draw. Layout takes effort, and the XML of a diagram diffs noisily
Markdown (template) PLANNED	Prose first: definitions, examples and rationale read naturally. Renders on every Git host, and diffs well	Structure by headings and tables, which are easy to break. Tables of many attributes are unwieldy
Java or Groovy, construction in code (Chapter 7)	Whatever a program can compute: harvesters and projections build vocabularies this way	A program that builds a vocabulary, not a vocabulary: nobody reviews it as one

Vocabularies are also harvested from where they are already written down: Ecore, Java, XML Schema, JSON Schema and database metadata PLANNED. Most of a domain model exists somewhere already, and asking people to type it again is how modeling efforts die.

A team chooses its format per package. Imports resolve by namespace through the resource set, so a package written in the text syntax imports one kept in a spreadsheet the same way it imports `party.nmeta`: the URI map points the namespace to an `.xlsx` file instead.

12.2. Hybrid authoring

Formats can also share one package PLANNED. Entities and relationships go on a diagram, as boxes and connections, where their shape is seen; attributes go in a spreadsheet or Markdown, where there is room for many; and descriptions go wherever prose is easiest to write.

The diagram is the backbone, and the other files are found from it, in one of two ways:

- **Implied**, from the identifier or the path of a diagram element. The attributes of a box `Customer` on the page `Retail` are the sheet `Customer` of a workbook next to the diagram, or the table in `Retail/Customer.md`.
- **Explicit**, in a property of the diagram element, such as a workbook and a sheet, or a Markdown file, which wins over what is implied.

Descriptions follow the same principle. An element's description is the text on the diagram, such as a tooltip; or a file at the element's path, found by trying extensions in order, `Retail/Customer.md`, then `.adoc`, then `.html`; or a file named in a property of the element. A description in a file becomes the element's `docRef` in the core model, a document resolved relative to the model and rendered in its format, so it is not copied into the model at all.

For the bank, a hybrid package could look like this:

```
bank/
  bank.drawio              Product, Customer, Account, Transaction and Statement as boxes,
                           holders and accounts as one connection
  bank.xlsx                a sheet per class: one row per attribute, with type,
                           multiplicity and definition
  Account.md               the definition of an account, with examples
  Account/balance.md       how a balance is computed, for the people who audit it
```

What a hybrid package must do is report what does not line up: a box without a sheet, a sheet without a box, and a property naming a file that is not there. Those are warnings with locations, as the text syntax reports them, and the markers of the elements say which file each part came from.

12.3. Output formats

Format	Strengths	Weaknesses
AsciiDoc, then PDF PLANNED	A document with a version, pages and a table of contents, which can be sent, signed off, attached to a contract and archived. The same AsciiDoc also renders to HTML and EPUB, with the toolchain this book is built with	A snapshot: it does not change when the vocabulary does, so it needs the vocabulary's version on its cover. Not navigable the way a site is
A documentation site PLANNED	A page per element, with its documentation, its neighbors and a diagram of its neighborhood. Searchable, linked and always current	Needs hosting. A site is not something a partner signs
Markdown, as the glossary of Chapter 9	Renders on every Git host and in wikis, and is compact enough for the context of an agent	Little control over layout. One file grows long
Meta text (Chapter 8)	The canonical form: whatever a vocabulary came from, it can be reviewed and compared as text	For developers
Ecore and Xcore, **TypeScript**, **JSON Schema** (Chapter 9) PLANNED	Types and schemas that cannot drift from the vocabulary: Ecore for EMF tools and generated Java, TypeScript for browsers, JSON Schema for validation and the tools of agents	For developers and agents, not for people reading about the business

Diagrams are outputs too: a class and its neighbors, laid out and rendered with the runtime's diagram models, as a page of a site or a figure of a PDF.

12.4. Who uses which

Who	Authors in	Reads
Developers	Meta text: it is reviewed like code, next to the code	Meta text, generated types and schemas
Architects and modelers	Draw.io, with attributes in a spreadsheet or in Markdown	The documentation site and its diagrams
Business owners of the vocabulary, such as product managers and data stewards	Excel and Markdown, where the vocabulary often lives already	The documentation site, and PDF for sign-off
Partners, auditors and regulators	Nothing: they receive the vocabulary	PDF: a versioned document that can be attached to an agreement
Agents	Proposals for review, in Meta text or Markdown, for a person to accept	Markdown and JSON Schema in their context, and Meta text

The columns differ for a reason. The people who know what the words mean are rarely the people who write them in a precise notation, and the people who need the vocabulary most, outside the organization, should never have to open an authoring tool at all. One model in the middle is what lets each of them use the format that suits them.

Until the spreadsheets, diagrams and Markdown are here, business owners take part through an assistant (Chapter 11). The weakness of the text syntax, a language to learn, then falls on the assistant, and its strengths, a precise file that diffs well and that a build can check, remain.

Chapter 13. Looking ahead

Formats come first, in Chapter 12. What follows are transformations of vocabularies, from Meta to Meta, and the dimensions a vocabulary varies in.

13.1. Flattening

Inheritance is how a model is authored. It is not always how it is consumed: a TypeScript interface, a JSON Schema for a tool call and a summary handed to an agent all read better as a flat class, with every inherited feature and operation inlined.

Flattening is a transformation from Meta to Meta **PLANNED**, not a trick inside each emitter: the input is a package, the output is a package whose classes declare everything they inherited, and every copied feature carries a marker back to the class that declared it. It is testable by comparison, reusable by every projection, and auditable.

Flattening is also where composition stops being free:

- Most encounters of the same name are not clashes. A feature reached twice through a diamond is one feature, and an operation declared again with the same signature is an override. Deduplicate by identity, not by name.
- The same name with different declarations is a clash, resolved by renaming, deterministically: the most derived declaration keeps the name, and the rename is recorded on the feature.
- Renaming happens in Meta, not downstream, because TypeScript refuses an interface that extends two interfaces declaring a member incompatibly, and a strict tool schema has no room for the composition that would otherwise carry it.

What a cut must not do is erase distinctions: removing an ancestor's name is recoverable by widening the cut, while replacing a class by its ancestor removes a distinction nothing can recover.

13.2. Variability

One vocabulary rarely means the same thing everywhere. A bank in the United States and Canada has two currencies, several jurisdictions, French as a legal requirement for consumer contracts in Quebec and a customer's preference elsewhere, and state rules, such as California's privacy law, over federal ones. Jurisdiction, currency, language and legal entity do not line up: Canadian customers commonly hold accounts in US dollars, so currency is not jurisdiction.

A fork per variant drifts within a release. Instead, classes, features and literals will be situated in coordinates, such as jurisdiction, language, time, product tier and audience **PLANNED**. One vocabulary then carries its variants together, the elements that differ marked with where and when they apply, and generating for a coordinate emits what applies there and nothing else: a product line, not a fork. Two operations will do it, and they are different on purpose:

- **Restriction** drops what does not apply to a region and keeps the vocabulary variable in every dimension it still spans. It is set restriction, so restricting to Ontario and then adding Quebec gives what restricting to both would.
- **Specialization** resolves a vocabulary at a point, such as Quebec, in French, as of a date, into a smaller vocabulary for one audience: the glossary a Quebec branch prints, or the schema a provincial integration validates against.

A resolved vocabulary records the point it was resolved at, so that a PDF says it is the Quebec French edition as of a date instead of merely being one, and can be compared with a sibling variant or resolved further.

13.3. Occupancy

Variability needs a way to say "this is true here, then, in that range" once, not five times: for jurisdictions, periods, versions, languages and product tiers alike. That is occupancy **PLANNED**. An element occupies a region of a space of dimensions, the region being a union of boxes, each a point or a range per dimension, and a dimension that is not mentioned is unconstrained, not unknown. Time is one dimension among others, with the best tooling and no special treatment.

The text syntax is ready for it. A facet attaches an occupancy to any element:

```
/** In Quebec, consumer contracts are in French unless the customer asks otherwise. */
@occupancy.Presence { jurisdiction: "CA-QC" }
```

```
string contractLanguage = "fr"
```

The bank's vocabulary in Chapter 8 says exactly that today, and the reader records the facet as an annotation, with a warning that its type is not available. When the occupancy model is, the same text is checked against it, and restriction and specialization become transformations from Meta to Meta, testable by comparison like flattening.

13.4. Kinds and types

A type, a Meta class or an Xcore class, fixes structure: which features an element has. The runtime's core model is gaining kinds, which say what an element is for regardless of its type. Generated Meta documentation will explain the difference, and the kind model comes first **PLANNED.**

The bank's vocabulary already leans that way: a checking account product is an instance of `Product`, not a class, so a jurisdiction's variant of a product is data, not a change to the vocabulary.

Appendix A: Reference

A.1. Classifiers

Classifier	What it is
`ModelElement`	Abstract base of Meta's elements, on the core model's `NamedElement`; contains annotations
`Annotation`	Details keyed by a source URI
`Package`	A namespace of classifiers with a namespace URI, a prefix and a version; the unit of publication
`Classifier`	Abstract base of classes, data types and enumerations; contains type parameters
`Class`	A type with features and operations, possibly abstract, with supertypes
`DataType`	A named scalar type, with its representation and source type
`Enum`, `EnumLiteral`	An enumeration and its literals, with serialized values and ordinals
`TypedElement`	Abstract: a type and a multiplicity
`Feature`	Abstract: state, possibly derived or read-only, with a default value
`Attribute`	A feature of a primitive or data type, possibly identifying
`Reference`	A feature of a class type: containment, opposite and keys
`Operation`, `Parameter`	A signature of behavior, and its parameters
`TypeParameter`	A type parameter with a bound and a default
`TypeReference` and its subclasses	A type in a type position (Chapter 6)
`Object`	The reflective instance of a class; operations declared, not implemented
`IncomingReference`	Who references an object, through what, and where

A.2. Text syntax

Written	Read as
`package bank namespace "…" prefix "bank" version "…"`	`Package`, its name, namespace, prefix and version
`import "<namespace>" as party`	`party.Name` is a `ClassifierReference` with that namespace and classifier name
`annotation "<source>" as Nasdanika`, then `@Nasdanika(key = "value")`	`Annotation` with the source and details
`@occupancy.Presence { jurisdiction: "CA-QC" }`	A facet, recorded as an `Annotation` for now (Section 8.6)
`/** … */`	The documentation of the element that follows, Markdown
`abstract class Name<T> extends A, B { … }`	`Class`
`type Name as string wraps java.util.Currency`	`DataType`, its representation and source type
`enum Name { A, B as "b" = 3 }`	`Enum` and `EnumLiteral`s, their values and ordinals
`id string[1] code`	`Attribute`, identifying
`refers Product[1] product`, `contains Statement[*] statements`	`Reference`, containment for `contains`
`derived`, `readonly`, `unordered`, `unique`	The flags of a feature
`opposite holders`, `keys transactionId`	The opposite and keys of a reference
`= "fr"`	The default value
`op Amount balanceAt(date day, string… codes) throws E`	`Operation`, `Parameter`s, variadic, exceptions
`A | B`, `A & B`, `Map<K, ? extends V>`, text in backticks	Union, intersection, type arguments and wildcards, opaque type
`^class`	A keyword used as a name

A.3. Primitive kinds

`STRING`, `NUMBER`, `BOOLEAN`, `BIGINT`, `DATE`, `TIME`, `DATE_TIME`, `DURATION`, `BINARY`, `ANY`, `NULL`, `VOID`.

A.4. Multiplicity

lowerBound	upperBound	**Meaning**
0	1	Optional, single-valued: the default
1	1	Required
0	-1	Any number
1	-1	At least one

Appendix B: Building this book

This book is AsciiDoc in `models/meta/model/src/docs/asciidoc`, built with the model after its tests have passed:

```
mvn -pl models/meta/model -am verify                          # the model, its tests and this
book
mvn -pl models/meta/model verify -Dzephyr.editor-notes=false  # without editor's notes
mvn -pl models/meta/model verify -Dasciidoctor.skip=true      # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the model's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/models/meta/tests/book`, and of the module declaration. A listing line holds at most 96 characters, and a warning fails the build.

The glossary in Chapter 9 is `library.md` in `src/test/resources/org/nasdanika/sdk/runtime/models/meta/tests/book`, which `MetaBookTests` compares with what the projection writes. Likewise, `TextSyntaxTests` compares the bank's glossary with `bank.md` and the library written as text with `library.nmeta`, in the same directory as `bank.nmeta` and `party.nmeta`. A change to a vocabulary, the projection or the writer changes them; run the tests, review the difference, and update the files.

The agent skill of Chapter 11 is in `src/main/skill/nasdanika-meta`. `SkillTests` compares its examples with the files above, leaving out the include tags, so a change to one of those files is copied to the skill too. It also checks that the lines of the skill's read-back examples are in `bank.nmeta`, and that the starter project writes the glossary where the skill says it is. It does not run the starter project, which needs the model installed; to try it, run `mvn install` on the model, copy the project somewhere, add a vocabulary, and run `mvn -q test`.

The parser of the text syntax is generated from `src/main/grammar/Meta.g4` into `src-gen` when the model is built, and committed with the Xcore-generated sources. A grammar warning fails the build.

Appendix C: Glossary

Agent skill

A directory of instructions that an AI assistant loads when a task calls for them. Meta's, `nasdanika-meta`, teaches an assistant to write and validate vocabularies.

Annotation

Details keyed by the URI of their owner, carrying what a source expressed and Meta does not.

Bounded context

In Domain-Driven Design, a boundary within which a model and its words have one meaning.

Facet

Typed data about an element, which another model defines: a role, an owner, an occupancy. Written `@Type { property: value }` in the text syntax.

Flattening

A transformation of a package into one whose classes declare everything they inherit.

Harvester

A loader that reads a vocabulary from where it is already written down, such as a database schema or an XML Schema.

Hybrid authoring

One vocabulary in several formats: entities and relationships on a diagram, attributes in a spreadsheet or Markdown, descriptions in files found from the diagram.

Identifying attribute

An attribute whose value identifies an instance, and which references may cite.

Metacircular

Able to describe itself: Meta's package of classes can be written in Meta.

Occupancy

Where and when an element applies: a region of a space of dimensions, such as jurisdiction, time and language.

Projection

A transformation of a vocabulary into a form a consumer needs: a glossary, an Ecore package, TypeScript, a JSON Schema.

Read-back

Plain statements of the rules a vocabulary's structure holds, which an assistant gives the people who own the words, the guessed ones first, so that they can correct them.

Published language

In Domain-Driven Design, a documented, shared vocabulary that bounded contexts translate into and out of.

Subject-matter expert

Someone who knows what the words of a domain mean, and need not know any notation: the person who judges a vocabulary's definitions and rules.

Text syntax

Meta's own notation, in `.nmeta` files: metamodels written the way Xcore writes Ecore models.

Validator

`MetaValidator`, which checks the `.nmeta` files of a directory together, and reports problems as a compiler does. It checks structure, not meaning.

Variability

How a vocabulary differs by jurisdiction, language, time or audience, carried in one vocabulary rather than in forks.

Vocabulary

A Meta package: the terms of a domain, their definitions, types and relations.